A Practical Guide to Diffusion model

TOPIC -- DIFFUSION MODEL

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\, x_{t-1}, \beta_t I\right)$$

-- 01 --

x t – d t = x t + 1 2 β (t) x t d t + β (t) f θ (x t , t) d t + β (t) d W t {\displaystyle x_{t-dt}=x_{t}+{\frac {1}{2}}\beta (t)x_{t}dt+\beta (t)f_{\theta }(x_{t},t)dt+{\sqrt {\beta (t)}}dW_{t}}

-- 02 --

Since the diffusion model is a general method for modelling probability distributions, if one wants to model a distribution over images, one can first encode the images into a lower-dimensional space by an encoder, then use a diffusion model to model the distribution over encoded images. Then to generate an image, one can sample from the diffusion model, then use a decoder to decode it into an image. The encoder-decoder pair is most often a variational autoencoder (VAE).

-- 03 --

L s i m p l e , t = E x 0 , x t ~ q [■ ■ θ (x t , t) – x t – α ¯ t x 0 σ t ■ 2] = E x t ~ q , x 0 ~ q (· | x t) [■ ■ θ (x t , t) – x t – α ¯ t x 0 σ t ■ 2] {\displaystyle L_{simple,t}=E_{x_{0},x_{t}\sim q}\left[\left\|\epsilon _{\theta }(x_{t},t)-{\frac {x_{t}-{\sqrt {{\bar {\alpha }}_{t}}}x_{0}}{\sigma _{t}}}\right\|^{2}\right]=E_{x_{t}\sim q,x_{0}\sim q(\cdot |x_{t})}\left[\left\|\epsilon _{\theta }(x_{t},t)-{\frac {x_{t}-{\sqrt {{\bar {\alpha }}_{t}}}x_{0}}{\sigma _{t}}}\right\|^{2}\right]}

-- 04 --

Classifier-free guidance (CFG) If we do not have a classifier p (y | x) {\displaystyle p(y|x)} , we could still extract one out of the image model itself:

-- 05 --

Diffusion model For generating images by DDPM, we need a neural network that takes a time t {\displaystyle t} and a noisy image x t {\displaystyle x_{t}} , and predicts a noise ■ θ (x t , t) {\displaystyle \epsilon _{\theta }(x_{t},t)} from it. Since predicting the noise is the same as predicting the denoised image, then

subtracting it from x t {\displaystyle x_{t}} , denoising architectures tend to work well. For example, the U-Net, which was found to be good for denoising images, is often used for denoising diffusion models that generate images. For DDPM, the underlying architecture ("backbone") does not have to be a U-Net. It just has to predict the noise somehow. For example, the diffusion transformer (DiT) uses a Transformer to predict the mean and diagonal covariance of the noise, given the textual conditioning and the partially denoised image. It is the same as standard U-Net-based denoising diffusion model, with a Transformer replacing the U-Net. Mixture of experts-Transformer can also be applied. DDPM can be used to model general data distributions, not just natural-looking images. For example, Human Motion Diffusion models human motion trajectory by DDPM. Each human motion trajectory is a sequence of poses, represented by either joint rotations or positions. It uses a Transformer network to generate a less noisy trajectory out of a noisy one.

-- 06 --

Backward diffusion process If we have solved ρ t {\displaystyle \rho _{t}} for time t ∈ [0 , T] {\displaystyle t\in [0,T]} , then we can exactly reverse the evolution of the cloud. Suppose we start with another cloud of particles with density ν 0 = ρ T {\displaystyle \nu _{0}=\rho _{T}} , and let the particles in the cloud evolve according to

-- 07 --

Sample (x 0 , z 0 , c) {\displaystyle (x_{0},z_{0},c)} , where x 0 {\displaystyle x_{0}} is the high-resolution image, z 0 {\displaystyle z_{0}} is the same image but scaled down to a low-resolution, and c {\displaystyle c} is the conditioning, which can be the caption of the image, the class of the image, etc. Sample two white noises ■ x , ■ z {\displaystyle \epsilon _{x},\epsilon _{z}} , two time-steps t x , t z {\displaystyle t_{x},t_{z}} . Compute the noisy versions of the high-resolution and low-resolution images: { x t x = α ¯ t x x 0 + σ t x ■ x z t z = α ¯ t z z 0 + σ t z ■ z {\displaystyle {\begin{cases}x_{t_{x}}&={\sqrt {{\bar {\alpha }}_{t_{x}}}}x_{0}+\sigma _{t_{x}}\epsilon _{x}\\z_{t_{z}}&={\sqrt {{\bar {\alpha }}_{t_{z}}}}z_{0}+\sigma _{t_{z}}\epsilon _{z}\end{cases}}} . Train the denoising network to predict ■ x {\displaystyle \epsilon _{x}} given x t x , z t z , t x , t z , c {\displaystyle x_{t_{x}},z_{t_{z}},t_{x},t_{z},c} . That is, apply gradient descent on θ {\displaystyle \theta } on the L2 loss ■ ■ θ (x t x , z t z , t x , t z , c) − ■ x ■ 2 2 {\displaystyle \|\epsilon _{\theta }(x_{t_{x}},z_{t_{z}},t_{x},t_{z},c)-\epsilon _{x}\|_{2}^{2}} .

-- 08 --

Denoising Diffusion Implicit Model (DDIM) The original DDPM method for generating images is slow, since the forward diffusion process usually takes T ~ 1000 {\displaystyle T\sim 1000} to make the distribution of x T {\displaystyle x_{T}} to appear close to Gaussian. However this means the backward diffusion process also take 1000 steps. Unlike the forward diffusion process, which can skip steps as x t | x 0 {\displaystyle x_{t}|x_{0}} is Gaussian for all t ≥ 1 {\displaystyle t\geq 1} , the backward diffusion process does not allow skipping steps. For example, to sample x t – 2 | x t – 1 ~ N (μ θ (x t – 1 , t – 1) , Σ θ (x t – 1 , t – 1)) {\displaystyle x_{t-2}|x_{t-1}\sim {\mathcal {N}}(\mu _{\theta }(x_{t-1},t-1),\Sigma _{\theta }(x_{t-1},t-1))} requires the model to first sample x t – 1 {\displaystyle x_{t-1}} . Attempting to directly sample x t – 2 | x t {\displaystyle x_{t-2}|x_{t}} would require us to marginalize out x t – 1 {\displaystyle x_{t-1}} , which is generally intractable. DDIM is a method to take any model trained on DDPM loss, and use it to sample with some steps skipped, sacrificing an adjustable amount of quality. If we generate the Markovian chain case in DDPM to non-Markovian case, DDIM corresponds to the case that the reverse process has variance equals to 0. In other words, the reverse process (and also the forward process) is deterministic. When using fewer sampling steps, DDIM outperforms DDPM. In detail, the DDIM sampling method is as follows. Start with the forward diffusion process x t = α ¯ t x 0 + σ t ■ {\displaystyle x_{t}={\sqrt {{\bar {\alpha }}_{t}}}x_{0}+\sigma _{t}\epsilon } . Then, during the backward denoising process, given x t , ■ θ (x t , t) {\displaystyle x_{t},\epsilon _{\theta }(x_{t},t)} , the original data is estimated as x 0 ′ = x t – σ t ■ θ (x t , t) α ¯ t {\displaystyle x_{0}'={\frac {x_{t}-\sigma _{t}\epsilon _{\theta }(x_{t},t)}{\sqrt {{\bar {\alpha }}_{t}}}}} then the backward diffusion process can jump to any step 0 ≤ s < t {\displaystyle 0\leq s<t} , and the next denoised sample is x s = α ¯ s x 0 ′ + σ s 2 – (σ s ′) 2 ■ θ (x t , t) + σ s ′ ■ {\displaystyle x_{s}={\sqrt {{\bar {\alpha }}_{s}}}x_{0}'+{\sqrt {\sigma _{s}^{2}-(\sigma '_{s})^{2}}}\epsilon _{\theta }(x_{t},t)+\sigma _{s}'\epsilon } where σ s ′ {\displaystyle \sigma _{s}'} is an arbitrary real number within the range [0 , σ s] {\displaystyle [0,\sigma _{s}]} , and ■ ~ N (0 , I) {\displaystyle \epsilon \sim {\mathcal {N}}(0,I)} is a newly sampled Gaussian noise. If all σ s ′ = 0 {\displaystyle \sigma _{s}'=0} , then the backward process becomes deterministic, and this special case of DDIM is also called "DDIM". The original paper noted that when the process is deterministic, samples generated with only 20 steps are already very similar to ones generated with 1000 steps on the high-level. The original paper recommended defining a single "eta value" η ∈ [0 , 1] {\displaystyle \eta \in [0,1]} , such that σ s ′ = η σ ~ s {\displaystyle \sigma _{s}'=\eta {\tilde {\sigma }}_{s}} . When η = 1 {\displaystyle \eta =1} , this is the original DDPM. When η = 0 {\displaystyle \eta =0} , this is the fully deterministic DDIM. For intermediate values, the process interpolates between them. By the equivalence, the DDIM algorithm also applies for

score-based diffusion models.